

UNIVERSIDAD COMPLUTENSE DE MADRID

MÁSTER EN CIENCIA DE DATOS, BIG DATA E INTELIGENCIA ARTIFICIAL



TRABAJO FIN DE MÁSTER

**Sistema Inteligente de Dynamic Pricing y
Optimización de Capacidad en el Transporte
Interurbano mediante Deep Reinforcement
Learning**

Inmaculada Osuna Muñoz

Septiembre de 2026

Resumen

Este Trabajo de Fin de Máster desarrolla una solución integral para la optimización dinámica de precios y la gestión de capacidad en la Alta Velocidad ferroviaria (caso de uso RENFE). El objetivo es maximizar los ingresos (*Revenue*) sobre un inventario perecedero y una demanda sensible al precio mediante Inteligencia Artificial.

La principal contribución del trabajo no reside sólo en entrenar un agente PPO, sino en diseñar un *pipeline* reproducible que integra datos reales, modelización de demanda, simulación secuencial, optimización mediante Aprendizaje por Refuerzo.

El proyecto se estructura en cuatro fases. Primero, se procesaron más de 38,7 millones de registros mediante un *pipeline* distribuido en **Apache Spark** (PySpark) [7], aplicando un particionado cronológico para evitar la fuga de información (*Data Leakage*).

Segundo, el problema se formuló como un Proceso de Decisión de Markov (MDP) [4], utilizando *Potential-Based Reward Shaping* [3] y un oráculo de Programación Dinámica.

En la tercera fase, se entrenó un agente de Aprendizaje por Refuerzo Profundo mediante la arquitectura *Proximal Policy Optimization* (PPO) [2], contrastándolo con un *baseline* supervisado basado en *Random Forest*. La validación mediante simulaciones de Monte Carlo muestra que PPO supera a la política estática en el 64,75 % de los escenarios analizados, especialmente en contextos de mayor precisión de demanda.

Finalmente, el modelo se llevó a una fase de productivización mediante la vectorización de su política en un archivo *JSON*. Esta política se acopló a una aplicación web (integrada con archivos `index` y `main`, versionada en GitHub y desplegada en Vercel) que permite consultar la política aprendida para las rutas disponibles en los conjuntos de entrenamiento y validación, mientras que la evaluación final se realizó sobre las 36 rutas presentes en el conjunto de test, incorporando herramientas de Inteligencia Artificial Explicable (*XAI*) y mecanismos de gobernanza mediante *Guardrails*.

Palabras clave: Dynamic Pricing, Deep Reinforcement Learning, Proximal Policy Optimization, Big Data, PySpark, Machine Learning, MLOps, Revenue Management.

Índice

Resumen Ejecutivo	1
Acceso al Material Complementario	4
1. Introducción y contextualización del problema	5
1.1. Naturaleza microeconómica de la Alta Velocidad	5
1.2. El reto del inventario estrictamente perecedero	5
1.3. De la tarifa estática al <i>Dynamic Pricing</i>	5
1.4. Objetivos del TFM y alineación curricular	6
2. Estado del arte: Evolución analítica del pricing	7
2.1. Heurísticas clásicas y el modelo EMSRb	7
2.2. Machine Learning supervisado: El paradigma Predict-then-Optimize	7
2.3. Justificación tecnológica: La miopía del ML clásico y la recompensa diferida	8
3. Desarrollo metodológico y pipeline de datos	9
3.1. Ingeniería de datos masivos	10
3.1.1. Orquestación y computación en Apache Spark	10
3.1.2. Particionado temporal y prevención del Data Leakage	10
3.2. Alcance e interpretación del entorno de simulación	11
3.3. Formulación del Proceso de Decisión de Markov (MDP)	12
3.3.1. Configuración del Espacio de Estados y Acciones	12
3.3.2. Estocasticidad de transición y Potential-Based Reward Shaping . .	12
3.4. Metodología de entrenamiento y ajuste del agente	14
3.4.1. Formulación del Baseline predictivo (Random Forest)	14
3.4.2. Arquitectura y exploración del Agente PPO	14
4. Evaluación, resultados y discusión analítica	16
4.1. Comparación cuantitativa de los modelos (Evidencia de Miopía)	16
4.2. Discusión crítica y análisis de sensibilidad por escenarios	18
5. Despliegue funcional, MLOps y Gobernanza	19
5.1. Inferencia serializada mediante búsqueda en memoria $\mathcal{O}(1)$	19
5.2. Arquitectura Frontend, XAI y Guardrails (Vercel)	20
6. Conclusiones y futuras líneas de investigación	21

A. Código Completo del Proyecto	23
A.1. Importación de Librerías y Configuración	23
A.2. Ingesta y ETL Masivo con PySpark	24
A.3. Entorno de Simulación (Gymnasium) y Programación Dinámica	27
A.4. Baseline de Machine Learning (Random Forest)	30
A.5. Entrenamiento del Agente PPO y Callbacks	32
A.6. Validación de Negocio (Monte Carlo) y Generación de Gráficos	34
A.7. Análisis de la Política, Heatmap PPO y Exportación MLOps	36
A.8. Arquitectura Guardrails y Alertas en Vanilla JS	38

Acceso al Material Complementario

Para facilitar la reproducibilidad y permitir la auditoría de los desarrollos, el código fuente completo se documenta en el **Apéndice**. Adicionalmente, se facilitan los siguientes recursos alojados en la nube:

- **Repositorio de Código (GitHub):** [Enlace al repositorio](#)

*Contiene el pipeline de PySpark, los cuadernos analíticos (.ipynb), el entorno **Gymnasium**, los archivos del backend/frontend web y los scripts de entrenamiento.*

- **Vídeo de Presentación:** [Enlace al vídeo](#)

Documento audiovisual de un máximo de 5 minutos que resume el enfoque de negocio, las decisiones metodológicas, el análisis estadístico y las principales conclusiones.

- **Aplicación Web:** [Enlace a la aplicación web](#)

Herramienta Front-End interactiva alojada en Vercel para auditar las predicciones del modelo, simular variables del entorno y observar los mapas de calor tarifarios en tiempo real.

Asimismo, se han concedido los permisos necesarios para la visualización del repositorio privado.

1. Introducción y contextualización del problema

1.1. Naturaleza microeconómica de la Alta Velocidad

El sector ferroviario opera bajo una estructura de costes muy particular. Una parte importante de estos costes es fija o se encuentra comprometida a largo plazo: la compra y amortización de los trenes, los cánones asociados al uso de la infraestructura y determinados gastos operativos, como la tripulación, el mantenimiento o la energía. Estos costes no varían proporcionalmente con el número de pasajeros que ocupan el tren.

Por ello, que un tren circule con un 20% de ocupación o prácticamente lleno no modifica de forma significativa una parte importante de sus costes de operación. Como contrapartida, el **coste marginal** de transportar a un pasajero adicional cuando existen plazas disponibles puede ser relativamente reducido.

1.2. El reto del inventario estrictamente perecedero

Viajar en tren es un ejemplo de actividad con **inventario perecedero** (tiene vida útil limitada y puede deteriorarse, caducar o perder valor con el paso del tiempo). Las plazas disponibles no pueden almacenarse para ser vendidas posteriormente.

En el momento en que el tren parte, cualquier asiento que permanezca vacío pierde la posibilidad de generar ingresos. Esta característica genera un **coste de oportunidad** y hace relevante la gestión de la capacidad disponible [6].

La combinación de costes fijos elevados, capacidad limitada y perecibilidad del inventario justifica la aplicación de técnicas de *Revenue Management*, cuyo objetivo es maximizar el ingreso obtenido a partir de la capacidad disponible [6].

1.3. De la tarifa estática al *Dynamic Pricing*

Las empresas ferroviarias, tradicionalmente, han usado estructuras tarifarias rígidas, condicionadas por factores como la temporada, el tipo de servicio o la antelación de compra. Sin embargo, la liberalización del mercado europeo y la entrada de nuevos operadores han incrementado la competencia y la presión sobre los márgenes.

En este contexto, los clientes pueden comparar diferentes alternativas en tiempo real mediante plataformas digitales. Los precios estáticos presentan dificultades para responder a variaciones de la demanda y para gestionar las plazas disponibles.

El ***Dynamic Pricing*** permite modificar los precios en función de las condiciones observadas en cada momento. El objetivo es adaptar el nivel tarifario a la demanda esperada y a la disponibilidad de plazas, buscando un equilibrio entre volumen e ingreso.

1.4. Objetivos del TFM y alineación curricular

Este Trabajo de Fin de Máster tiene como propósito diseñar, programar y validar un sistema de fijación de precios empleando técnicas de Ciencia de Datos, Machine Learning y Aprendizaje por Refuerzo. El proyecto constituye no solo una aplicación práctica de los conocimientos adquiridos durante el **Máster en Ciencia de Datos, Big Data e Inteligencia Artificial** de la Universidad Complutense de Madrid [1], sino también el fruto de una extensa investigación personal, conectando cada fase del desarrollo con diferentes competencias:

1. **Ingeniería Big Data y Computación Distribuida:** desarrollo de arquitecturas de computación distribuida con *Apache Spark* [7] para procesar, limpiar y analizar más de 38,7 millones de registros transaccionales históricos.
2. **Machine Learning Supervisado:** construcción de un modelo predictivo base mediante algoritmos de ensamblaje (*Random Forest*), utilizado como referencia (*baseline*) para comparar diferentes estrategias de pricing.
3. **Aprendizaje por Refuerzo Profundo (DRL):** formulación del problema como un Proceso de Decisión de Markov (MDP) [4] y entrenamiento de un agente mediante un algoritmo Actor-Crítico, concretamente *Proximal Policy Optimization* (PPO) [2].
4. **Despliegue Funcional y MLOps:** integración del modelo en un entorno web completo (con despliegue en Vercel y código fuente en GitHub), incorporando herramientas de *Explainable AI* (XAI) y MLOps.

2. Estado del arte: Evolución analítica del pricing

La disciplina del *Dynamic Pricing* tiene sus raíces en la modelización matemática y la Investigación Operativa (*Operations Research*) [6]. Examinar la evolución de estas metodologías permite contextualizar la aplicación del Aprendizaje por Refuerzo al problema de optimización tarifaria.

2.1. Heurísticas clásicas y el modelo EMSRb

La investigación formal sobre el control de capacidades dio un paso importante en 1972 con la conocida **Regla de Littlewood** [8], orientada inicialmente a situaciones con dos clases tarifarias. La regla establece que una reserva de la clase de menor tarifa debe aceptarse cuando su ingreso es superior al ingreso esperado de reservar la plaza para una futura venta de una clase de tarifa superior:

$$p_2 \geq p_1 \cdot P(D_1 > x) \quad (2.1)$$

donde $P(D_1 > x)$ representa la probabilidad de que la demanda futura de la clase de tarifa alta (D_1) supere las plazas disponibles (x) [8]. Posteriormente, Belobaba generalizó esta lógica para sistemas con múltiples clases tarifarias mediante métodos como **EMSRb** (*Expected Marginal Seat Revenue*) [6].

Aunque estas técnicas tuvieron una gran importancia en el desarrollo del *Revenue Management*, se apoyan en determinados supuestos sobre la demanda y el comportamiento de los consumidores [6]. Cuando las condiciones reales se alejan de dichos supuestos, su capacidad de adaptación puede verse limitada.

2.2. Machine Learning supervisado: El paradigma Predict-then-Optimize

Con el aumento de la capacidad de almacenamiento y procesamiento, las empresas han incorporado modelos de *Machine Learning* supervisado para estimar la demanda.

Algoritmos como *Random Forest*, *Gradient Boosting* o redes neuronales pueden utilizar diferentes variables explicativas para estimar la demanda esperada $\hat{D}(p)$ ante una determinada tarifa p . Posteriormente, un componente optimizador puede seleccionar el precio que maximiza una función de ingreso estimada:

$$Revenue(p) = p \times \hat{D}(p). \quad (2.2)$$

2.3. Justificación tecnológica: La miopía del ML clásico y la recompensa diferida

El planteamiento del *Machine Learning* supervisado presenta una limitación en la gestión de inventarios perecederos [6]: la predicción se realiza para una situación concreta y no necesariamente tiene en cuenta las consecuencias de la decisión sobre el inventario futuro.

Los modelos predictivos tradicionales son «miopes». En este entorno hay secuencialidad, capacidad limitada y el inventario es perecedero; existe un constante *trade-off* entre vender ahora o esperar. Si un algoritmo supervisado detecta que aplicando un descuento agresivo hoy se venderán decenas de billetes, lo recomendará para maximizar el ingreso inmediato. Sin embargo, al hacerlo, agota antes de tiempo la capacidad, dejando sin plazas a viajeros que estaban dispuestos a pagar la tarifa máxima más tarde. Así, una decisión aparentemente rentable en el presente puede destruir el valor global del trayecto.

Para abordar este problema, debe formularse como un **control óptimo secuencial** mediante Aprendizaje por Refuerzo (*Reinforcement Learning*) [5].

El agente interactúa con un entorno, observa el estado actual, selecciona una acción y recibe una recompensa. La formulación se relaciona directamente con la **Ecuación de Bellman** [4]:

$$V^*(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right). \quad (2.3)$$

A través de esta ecuación, formalizada por Bellman [4] y ampliada en el marco del aprendizaje secuencial por Sutton y Barto [5], el agente de IA busca maximizar la *recompensa acumulada esperada* al finalizar la venta. El DRL aprende a equilibrar la estimulación de la demanda con la preservación del inventario, incorporando el valor futuro de las plazas restantes.

3. Desarrollo metodológico y pipeline de datos

El desarrollo de este trabajo se estructura en diferentes fases, siguiendo la secuencia lógica implementada en el Jupyter Notebook. Tal y como se ilustra esquemáticamente en la Figura 1, la arquitectura del sistema se articula en seis grandes etapas: partiendo de la ingesta y el procesamiento distribuido de los datos, el flujo avanza hacia la configuración del entorno de simulación y el entrenamiento de los modelos (tanto el baseline supervisado como el agente de IA autónoma), para culminar en la rigurosa validación estadística mediante Monte Carlo y la exportación de los resultados para su productivización.

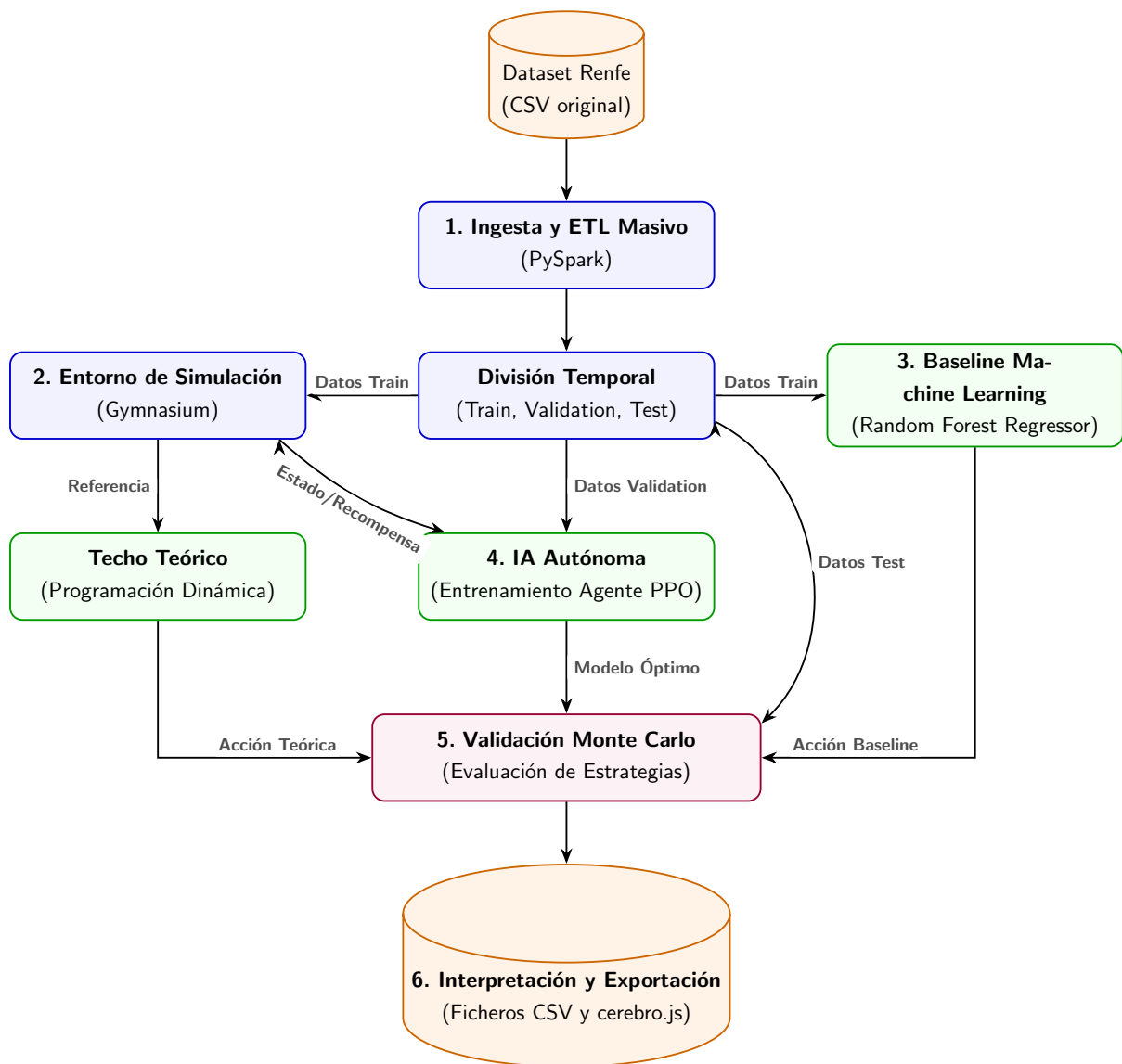


Figura 1: Diagrama de flujo de la metodología y arquitectura del sistema.

3.1. Ingeniería de datos masivos

La capacidad de generalización de cualquier arquitectura de aprendizaje automático depende en gran medida de la calidad, limpieza y estructuración de los datos utilizados durante el entrenamiento. En esta sección se detalla el procesamiento del *dataset*.

3.1.1. Orquestación y computación en Apache Spark

El conjunto de datos histórico transaccional constaba de un total de **38.753.060 registros únicos y 14 dimensiones**. El procesamiento de un volumen de esta magnitud genera limitaciones de memoria cuando se intenta cargar en estructuras locales mediante herramientas como **Pandas**. Por este motivo, el procesamiento se realizó mediante **Apache Spark** (PySpark) [7], aprovechando la evaluación perezosa (*lazy evaluation*) y el optimizador *Catalyst* para mejorar la eficiencia de la limpieza, transformación y agregación masiva.

Durante la fase de Análisis Exploratorio de Datos (EDA), se derivaron variables críticas como el cálculo de la antelación comercial (`lead_time_dias`), filtrando la ventana temporal al intervalo [0,30] días, concentrando el análisis en la operativa del entorno de simulación.

3.1.2. Particionado temporal y prevención del Data Leakage

En problemas temporales, evitar la fuga de información (*Data Leakage*) resulta fundamental para obtener una evaluación real. Por este motivo, se descartó la partición aleatoria estándar y se implementó un **particionado temporal cronológico** basado en la fecha de salida de los servicios. El código responsable de esta partición puede verse en el **Apéndice A.2**.

La partición se realizó cronológicamente sobre las fechas de salida, utilizando aproximadamente el 70 % de las fechas para entrenamiento, el 10 % para validación y el 20 % para test.

La Tabla 1 presenta un resumen de la distribución final de la información procesada.

Tabla 1: Tabla resumen de las dimensiones del Dataset

Característica	Valor
Nº total de registros brutos	38.753.060
Nº registros en Train	~13,4 millones (84 %)
Nº registros en Validation	~784 mil (5 %)
Nº registros en Test	~1,6 millones (11 %)
Nº rutas operativas evaluadas	56 (36 exclusivas en TEST)
Horizonte temporal (Lead Time)	30 días previos a la salida

3.2. Alcance e interpretación del entorno de simulación

Los datos históricos de antelación, rutas y tarifas son reales, pero **el entorno donde aprende PPO es simulado**. La demanda se aproxima mediante distribuciones de Poisson y supuestos elásticos exógenos. En consecuencia, el Oráculo de Programación Dinámica (DP) representa el comportamiento óptimo del simulador diseñado y no el techo de una solución del mundo real.

3.3. Formulación del Proceso de Decisión de Markov (MDP)

El problema se formuló como un Proceso de Decisión de Markov (MDP) [4], representado mediante la tupla $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ [5]. La implementación del entorno se realizó en Python utilizando la API de Gymnasium.

3.3.1. Configuración del Espacio de Estados y Acciones

Para facilitar el entrenamiento de la red neuronal y evitar gradientes inestables, las variables de entrada se normalizaron de forma estricta al intervalo $[0, 1]$:

$$S_t = \begin{bmatrix} \frac{t}{T_{\text{máx}}} \\ \frac{c_t}{C_{\text{máx}}} \\ \frac{p_{\text{base},t}}{P_{\text{escala}}} \end{bmatrix} \in [0, 1]^3 \quad (3.1)$$

Los componentes de la ecuación (3.1) representan la fracción de la ventana temporal restante, la **capacidad disponible** c_t en el estado t respecto al máximo del tren, y el precio base normalizado. El **espacio de acciones** se definió de forma discreta mediante nueve multiplicadores tarifarios:

$$A \in \{0,6, 0,7, 0,8, 0,9, 1,0, 1,1, 1,2, 1,3, 1,4\} \quad (3.2)$$

Estos límites se han definido para modelar una banda de fluctuación de precios en el sector ferroviario, abarcando desde un 40 % de descuento (0.6x) para estimular la demanda en escenarios valle, hasta un 40 % de recargo (1.4x) para maximizar la captura de excedente económico cuando existe alta demanda y escasez de plazas.

3.3.2. Estocasticidad de transición y Potential-Based Reward Shaping

El número de compras diarias se modeló mediante una **Distribución de Poisson**. La tasa de llegada esperada, λ , depende de la demanda histórica del ETL, modulada por el escenario de demanda y la elasticidad respecto al precio propuesto por el agente [6].

Para solucionar el problema de la asignación de crédito temporal (*Sparse Rewards*), se usó una función de recompensa basada en la técnica **Potential-Based Reward Shaping** formalizada por Ng, Harada y Russell [3]. Primero, se resolvió el techo teórico (*Upper Bound*) mediante un oráculo de Programación Dinámica (DP) calculando el valor exacto de la política óptima $V_{dp}(t, c)$ para cada estado definido por los días restantes t y la capacidad c , resolviendo recursivamente la Ecuación de Bellman tal y como se muestra en la ecuación (3.3):

$$\begin{aligned}
V_{dp}(t, c) = \max_{a \in \mathcal{A}} & \left(\sum_{d=0}^{c-1} P(D_a = d) \left(p_a \cdot d + V_{dp}(t-1, c-d) \right) \right. \\
& \left. + P(D_a \geq c) \left(p_a \cdot c + V_{dp}(t-1, 0) \right) \right)
\end{aligned} \tag{3.3}$$

donde D_a es la demanda estocástica modelada por una distribución de Poisson para una acción de precio p_a . Este valor óptimo calculado *offline* se integró posteriormente como función de potencial $\Phi(s) = V_{dp}(s)$ para guiar al agente DRL mediante recompensas densas e informativas.

Como se puede ver en el **Apéndice A.3**, la función **step** de Gymnasium hace la transición del entorno y calcula la recompensa conformada, lo cual cubre operativamente la dinámica estocástica expresada en la ecuación (3.3).

3.4. Metodología de entrenamiento y ajuste del agente

3.4.1. Formulación del Baseline predictivo (Random Forest)

Para comparar el rendimiento del agente DRL con un enfoque clásico supervisado, y analizar la posible limitación miope expuesta, se utilizó como *baseline* un modelo **Random Forest Regressor** (Scikit-Learn). A partir del *Train Set*, se construyó una muestra sintética de 50.000 observaciones. Ante cada estado del simulador, el árbol evalúa las nueve acciones tarifarias disponibles y selecciona aquella que maximiza el ingreso estimado puntual:

$$a^* = \arg \max_a \left(\hat{D}_a \cdot P_a \right). \quad (3.4)$$

Nota sobre el modelo base: El baseline Random Forest (cuyo código de creación se encuentra en el **Apéndice A.4**) se incluye como referencia de decisión miope y no como predictor supervisado de demanda real, ya que se entrena sobre ventas sintéticas generadas a partir del simulador. Su función principal es servir como comparación frente a una política que optimiza ingreso inmediato.

3.4.2. Arquitectura y exploración del Agente PPO

El agente principal se desarrolló mediante la arquitectura Actor-Crítico **Proximal Policy Optimization (PPO)** desarrollada por Schulman et al. [2], utilizando Stable-Baselines3 y redes ocultas de [128,128] neuronas. PPO utiliza una función objetivo recortada (*Clipped Surrogate Objective*) que limita el tamaño de las actualizaciones de la política para mejorar la estabilidad [2].

Para evitar que la IA converja antes de tiempo en una política subóptima por falta de exploración, se forzó la entropía en la función de pérdida. El código íntegro del entrenamiento se detalla en el **Apéndice A.5**.

La parametrización final, seleccionada sobre el entorno de validación sin contaminar el *Test Set*, se resume en la Tabla 2:

Tabla 2: Tabla de Hiperparámetros de la Red PPO

Hiperparámetro	Valor Configurado
Learning Rate	1×10^{-4}
Batch Size	256
N Steps	2048
Gamma (γ)	1.0
Entropía (<code>ent_coef</code>)	0.02
GAE Lambda (<code>gae_lambda</code>)	0.95
Reward Shaping Coef	0.02
Total Timesteps	200.000

4. Evaluación, resultados y discusión analítica

Evaluar el rendimiento de una arquitectura estocástica a partir de la curva de entrenamiento en entornos de simulación resulta insuficiente. Por ello, la evaluación de negocio se ejecutó utilizando el **Conjunto de Test** bajo una validación de escenarios. Se simularon 1,000 episodios virtuales mediante Monte Carlo por cada una de las 36 rutas operativas completas, combinando de forma cruzada múltiples niveles de elasticidad y choque de demanda. Esta exhaustiva malla experimental se traduce en la ejecución y evaluación masiva de **1.620.000 operaciones simuladas independientes**, permitiendo reforzar la robustez estadística de los resultados y reducir el efecto de la estocasticidad inherente del entorno.

4.1. Comparación cuantitativa de los modelos (Evidencia de Miopía)

Los resultados agregados de las simulaciones mediante Monte Carlo destaparon el comportamiento de cada arquitectura en entornos estocásticos. En esta sección se diferencia entre el **Ingreso Bruto** y el **Objetivo Medio**, métrica que el agente optimiza restando al ingreso la penalización corporativa impuesta por dejar asientos vacíos al partir el tren.

Tabla 3: Resultados Consolidados del Experimento (Conjunto de Test)

Estrategia	Objetivo Medio	Mejora vs Base	Ocupación Media	Multiplicador
<i>DP Oráculo (Techo)</i>	<i>5.794,97 €</i>	<i>+ 609,40 €</i>	<i>91,92 %</i>	<i>1.11x</i>
Agente DRL (PPO)	5.211,46 €	+ 25,89 €	85,08 %	0.99x
Heurística Programada	5.202,19 €	+ 16,62 €	88,41 %	0.95x
Política Estática (1.0x)	5.185,57 €	Base de referencia	86,79 %	1.00x
Random Forest (ML)	5.136,79 €	- 48,78 €	66,36 %	1.38x

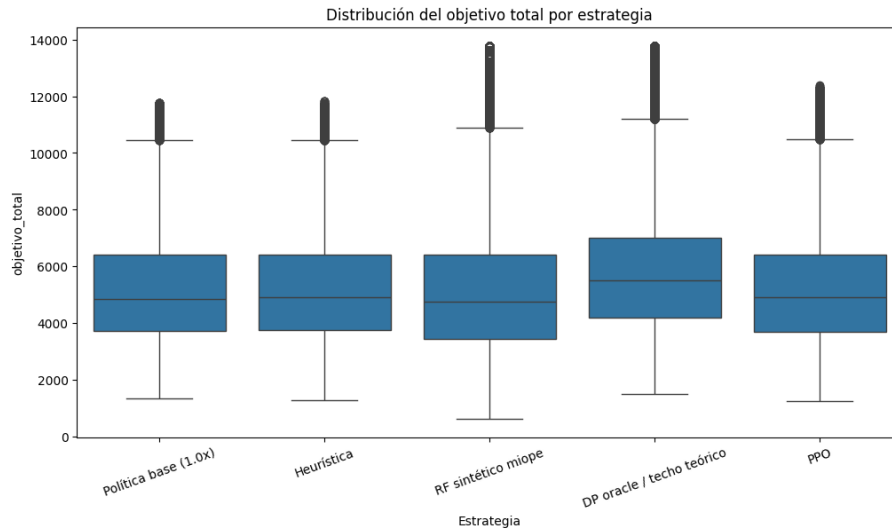


Figura 2: Distribución del Objetivo Total por estrategia (Boxplot). Se aprecia cómo PPO logra sostener el límite superior sin colapsar su rendimiento frente a la alta varianza del modelo miope.

Tal como se fundamentó, **los resultados obtenidos son consistentes con la hipótesis de una posible miopía operativa del enfoque supervisado**. Al centrarse en maximizar la estimación diaria del ingreso inmediato, el árbol de decisión elevó el multiplicador medio al límite de forma constante ($1,38x$). Esta agresividad le permitió igualar prácticamente el ingreso bruto nominal de PPO, pero provocó un daño logístico severo: **la ocupación de los trenes se desplomó al 66,36 %**. Al aplicar la función objetivo, que penaliza los trenes vacíos, el modelo supervisado cae a la última posición de eficiencia.

Por su parte, el **Agente PPO (DRL)** aprendió una política coherente con el equilibrio entre ingreso inmediato y preservación del valor futuro del inventario mediante la teoría de control secuencial [5]. Con un ajuste táctico del precio promedio a **0.99x**, logró el máximo Objetivo de Negocio de todas las estrategias aprendidas (5.211,46 €), sosteniendo una capacidad altamente eficiente.

La coincidencia acción a acción entre la política neuronal (PPO) y el Oráculo DP no es matemáticamente perfecta (Figura 3), lo cual era esperable. En determinadas regiones del espacio de estados pueden existir varias acciones con valores esperados próximos, por lo que dos políticas pueden diferir localmente en una matriz sin que ello implique una grave diferencia en el rendimiento económico agregado del viaje.

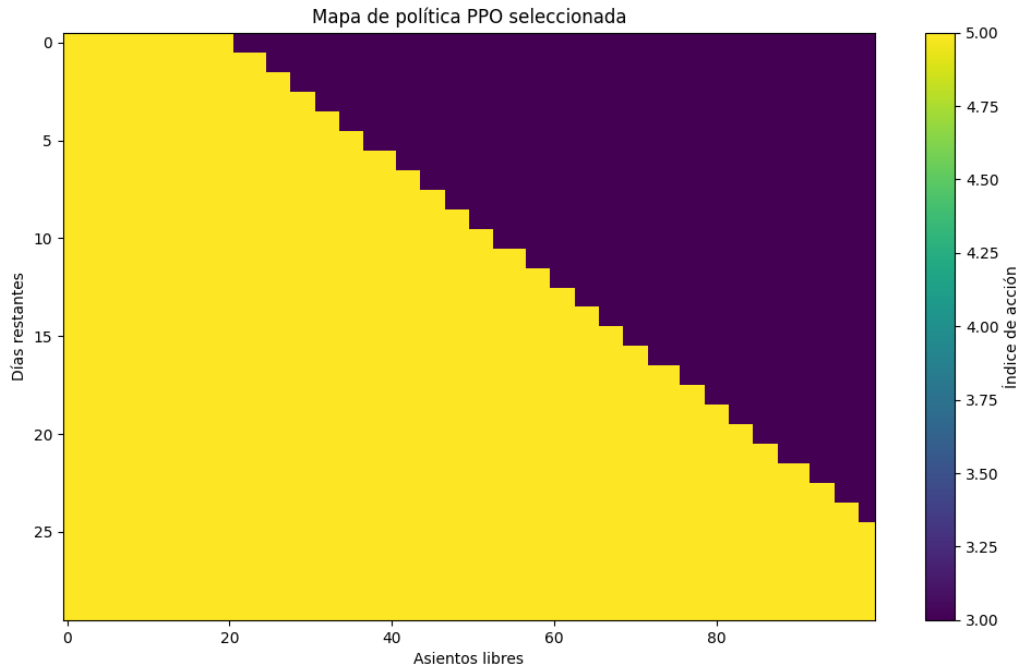


Figura 3: Mapas topológicos de política para una ruta dada. (Izquierda) Frontera de decisión estricta por Programación Dinámica. (Derecha) Lógica aprendida generalizada por la red neuronal PPO.

4.2. Discusión crítica y análisis de sensibilidad por escenarios

El análisis inferencial (*Bootstrapping*) evidenció que PPO superó a la política estática en el **64,75 %** de los escenarios según la métrica de objetivo definida.

Una discusión crítica obliga a analizar ¿cuándo funciona mejor y por qué pasó? Los datos evidencian que **el agente DRL sobresale en la gestión del estrés operativo**. En escenarios de alta demanda (1.2x), donde las reglas fijas o humanas suelen fracasar al liquidar el tren a tarifas excesivamente baratas y agotar la capacidad pronto, PPO encarece los asientos, protegiendo el inventario y obteniendo mejoras de hasta +116,35 € por trayecto en determinados contextos. Es aquí donde el agente aprende el valor futuro del asiento.

Por el contrario, ¿cuándo funciona peor y qué explica la brecha con DP? En escenarios depresivos, donde la demanda cae (0.8x) y el consumidor se vuelve muy sensible a las alteraciones de tarifa (alta elasticidad), el modelo PPO no logra encontrar la precisión del Oráculo para la estimulación promocional, generando pérdidas frente al sistema estático. Esta debilidad explica la brecha existente contra el DP y justifica la necesidad de una integración humana híbrida.

5. Despliegue funcional, MLOps y Gobernanza

El desarrollo de un modelo matemático carece de valor empresarial si no puede integrarse de forma segura y eficiente en los ecosistemas de la compañía, y si sus decisiones resultan inescrutables para los equipos humanos. Para evitar que el proyecto quede como un *Jupyter Notebook* aislado, el TFM culmina con el despliegue funcional de un *Dashboard*.

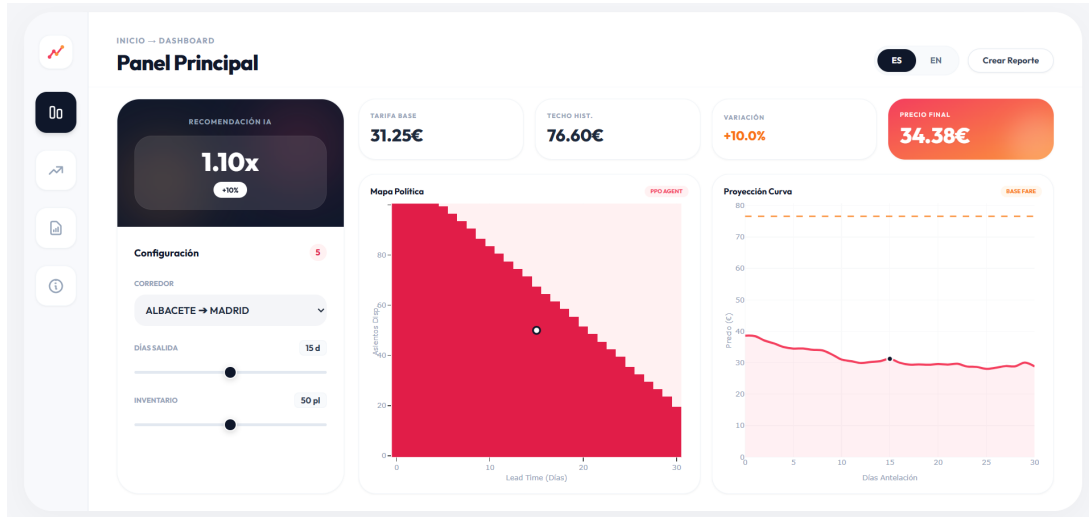


Figura 4: Interfaz del *Dashboard* interactivo. A la izquierda, el panel de configuración de parámetros del escenario. En el centro, el mapa de calor renderizado en tiempo real (XAI) que permite auditar visualmente las decisiones de la política neuronal para la ruta seleccionada. Además, en la botonera izquierda, se pueden auditar y comprobar los datos obtenidos junto con una explicación de lo realizado.

5.1. Inferencia serializada mediante búsqueda en memoria $\mathcal{O}(1)$

Lanzar inferencias contra una red neuronal profunda por cada consulta de billete en la web incrementaría la latencia y los costes de infraestructura de *cloud computing*. Sin embargo, dado que el espacio de estados operativo del modelo MDP está acotado (días de antelación y asientos de capacidad), el problema se resolvió con un enfoque **MLOps desacoplado**.

Mediante un proceso de cómputo *offline*, se forzó al agente entrenado a inferir todas y cada una de las combinaciones posibles de estado para todas las rutas. El resultado de la política neuronal se vectorizó en un diccionario comprimido. Para examinar la lógica de exportación JSON, ver el **Apéndice A.7**.

El frontend lee el archivo `cerebro.js` y se limita a realizar una búsqueda clave-valor en memoria del navegador con complejidad de tiempo $\mathcal{O}(1)$. Es importante precisar que esta complejidad constante se refiere a la consulta en producción por parte de la interfaz web; el cálculo integral de la política, por el contrario, requiere el coste computacional del entrenamiento y la inferencia exhaustiva previa del modelo.

5.2. Arquitectura Frontend, XAI y Guardrails (Vercel)

Para dotar de operatividad y proveer al equipo de negocio de una interfaz de auditoría en tiempo real, el repositorio se completó con un desarrollo *Frontend*, versionado en **GitHub** y desplegado sobre la infraestructura de **Vercel**.

La interfaz de usuario ha sido estilizada utilizando **Tailwind CSS** para asegurar su adaptabilidad (*responsive design*) y consta de cuatro vistas analíticas (Dashboard, Métricas, Explorador de Datos y Documentación Técnica). Tal y como se ilustra en la **Figura 4**, la plataforma incorpora tres elementos críticos para la confianza corporativa:

- **Inteligencia Artificial Explicable:** La inclusión de gráficos interactivos funcionan como un mecanismo de inspección e interpretación de la política. El Mapa de Calor permite al analista ver exactamente en qué frontera la IA decide encarecer el billete o aplicar rebajas.
- **Exportación:** La interfaz permite la exportación dinámica de las rutas a formato CSV.
- **Capa de Gobernanza (Guardrails):** Se programó una capa de seguridad sobre el modelo (ver **Apéndice A.8**). Si la IA recomienda un multiplicador que viola la banda tarifaria corporativa, el Guardarraíl intercepta la predicción, lanza una alerta visual en la UI ("Límite Aplicado") y trunca el precio, introduciendo una restricción determinista y auditable.

6. Conclusiones y futuras líneas de investigación

Los resultados obtenidos muestran que, bajo las condiciones y supuestos definidos en este estudio, la utilización de **Deep Reinforcement Learning** puede constituir una alternativa eficaz para la optimización dinámica de precios y la gestión de inventario perecedero en el transporte interurbano de Alta Velocidad [6, 5].

A través de la arquitectura PPO [2], el agente DRL mitigó la limitación de enfoque miope observada en el modelo supervisado, logrando armonizar la maximización del *Objetivo Comercial* con el mantenimiento de métricas de capacidad operativas altamente eficientes (superiores al 85 %), evidenciando su capacidad de interpretar la recompensa diferida a lo largo del horizonte temporal [5].

Implicación de negocio

Los resultados sugieren que estrategias de pricing adaptativas pueden capturar valor adicional frente a políticas fijas, especialmente en contextos con suficiente margen temporal y sensibilidad moderada de la demanda. No obstante, el sistema no debe desplegarse como un sistema completamente automatizado. El valor empresarial se alcanza desplegándolo como un Sistema de Apoyo a la Decisión Híbrido, donde reglas humanas gobiernan periodos valle y el PPO domina picos de demanda.

Limitaciones

El entorno de prueba es simulado, la demanda asume un comportamiento Poisson con una elasticidad exógena, se omite por completo la agresividad de la competencia del mercado libre, el RF evaluado es sintético, y la Programación Dinámica se establece como un techo del simulador, y no como una garantía del mundo físico real.

Reproducibilidad

La partición temporal, la fijación de semillas globales estocásticas en Gymnasium, y la exportación trazada del pipeline en Python aseguran una evaluación completamente reproducible y auditable en los repositorios facilitados.

Líneas de Trabajo Futuro

1. **Enriquecimiento del Espacio de Estados Exógeno:** Incorporar vectores adicionales de estado en la red neuronal obtenidos mediante *web-scraping*, tales como la tarifa en tiempo real ofrecida por competidores directos (Ouigo, Iryo).
2. **Transición hacia Espacios de Acciones Continuos:** Evolucionar del espacio discreto a un espacio de inferencia continua.
3. **Multi-Agent Reinforcement Learning:** Escalar el entorno Gymnasium para albergar múltiples inteligencias artificiales que representen a distintas compañías ferroviarias operando sobre la misma clientela.

Bibliografía

- [1] Universidad Complutense de Madrid (UCM) (2025–2026). *Máster en Ciencia de Datos, Big Data e Inteligencia Artificial: Materiales docentes, módulos de Deep Learning, Big Data y Machine Learning*. Universidad Complutense de Madrid.
- [2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347.
- [3] Ng, A. Y., Harada, D., & Russell, S. (1999). *Policy invariance under reward transformations: Theory and application to reward shaping*. Proceedings of the 16th International Conference on Machine Learning (ICML), 278–287.
- [4] Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.
- [5] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [6] Talluri, K. T., & Van Ryzin, G. J. (2004). *The Theory and Practice of Revenue Management*. Springer.
- [7] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). *Apache Spark: A Unified Engine for Big Data Processing*. Communications of the ACM, 59(11), 56–65.
- [8] Littlewood, K. (1972). *Forecasting and control of passenger bookings*. In Proceedings of the 12th AGIFORS Symposium, 95–117.

A. Código Completo del Proyecto

A continuación, se detalla la base de código íntegra ejecutada en el *Jupyter Notebook* original para garantizar el 100 % de la reproducibilidad del proyecto.

A.1. Importación de Librerías y Configuración

```

1      import json
2      import os
3      import random
4      import sys
5      import warnings
6      from pathlib import Path
7
8      import gymnasium as gym
9      import matplotlib.pyplot as plt
10     import numpy as np
11     import pandas as pd
12     import pyspark
13     import sklearn
14     import stable_baselines3
15     from gymnasium import spaces
16     from scipy.stats import poisson
17     from sklearn.ensemble import RandomForestRegressor
18     from stable_baselines3 import PPO
19     from stable_baselines3.common.callbacks import EvalCallback
20     from stable_baselines3.common.env_checker import check_env
21     from stable_baselines3.common.monitor import Monitor
22     from stable_baselines3.common.utils import set_random_seed
23
24     from pyspark.sql import SparkSession
25     from pyspark.sql.functions import (
26         avg, col, count, countDistinct, datediff, expr, lit, round as spark_round,
27         to_date
28     )
29
30     SEED = 42
31     MAX_DIAS = 30
32     CAPACIDAD = 100
33
34     ELASTICIDAD_NOMINAL = -1.2
35     FACTOR_DEMANDA_NOMINAL = 1.0
36     PENALIZACION_VACIOS = 0.10
37
38     random.seed(SEED)
39     np.random.seed(SEED)
40     set_random_seed(SEED)
41
42     BASE_DIR = Path.cwd()
43     RUTA_CSV = BASE_DIR / "thegurus-opendata-renfe-trips.csv"
44
45     warnings.filterwarnings("ignore", category=UserWarning, module="gymnasium")
46     warnings.filterwarnings("ignore", category=DeprecationWarning, module="pyspark")

```

A.2. Ingesta y ETL Masivo con PySpark

```

1      COL_FECHA_COMPRA = "insert_date"
2      COL_FECHA_SALIDA = "departure"
3      COL_PRECIO = "price"
4      COL_ORIGEN = "origin"
5      COL_DESTINO = "destination"
6      COL_TIPO_VEHICULO = "vehicle_type"
7      COL_CLASE = "vehicle_class"
8      COL_TARIFA = "fare"
9
10     spark = SparkSession.builder.appName("TFM_Renfe_DynamicPricing_DRL").getOrCreate
11     ()
12
13     df_raw = (
14         spark.read.format("csv")
15         .option("header", "true")
16         .option("inferSchema", "true")
17         .option("quote", "'")
18         .option("escape", "'")
19         .option("multiline", "true")
20         .load(str(RUTA_CSV))
21     )
22
23     from pyspark.sql.functions import translate, upper
24
25     df_clean = (
26         df_raw
27         .withColumn(COL_ORIGEN, translate(upper(col(COL_ORIGEN)), 'ÁÉÍÓÚ', 'AEIOU'))
28         .withColumn(COL_DESTINO, translate(upper(col(COL_DESTINO)), 'ÁÉÍÓÚ', 'AEIOU'))
29         .withColumn("fecha_compra_dt", to_date(col(COL_FECHA_COMPRA)))
30         .withColumn("fecha_salida_dt", to_date(col(COL_FECHA_SALIDA)))
31         .withColumn("lead_time_dias", datediff(col("fecha_salida_dt"), col("
32 fecha_compra_dt"))))
33         .filter(col("lead_time_dias").between(0, MAX_DIAS))
34         .filter(col(COL_PRECIO).isNotNull() & (col(COL_PRECIO) > 0))
35         .dropna(subset=[COL_ORIGEN, COL_DESTINO, COL_TIPO_VEHICULO, COL_CLASE, COL_TARIFA
36 , "fecha_salida_dt", "fecha_compra_dt"])
37         .cache()
38     )
39
40     fechas_salida = [
41         fila["fecha_salida_dt"]
42         for fila in (df_clean.select("fecha_salida_dt").where(col("fecha_salida_dt").
43 isNotNull()).distinct().orderBy("fecha_salida_dt").collect())
44     ]
45
46     idx_fin_train = max(1, int(0.70 * len(fechas_salida)))
47     idx_fin_val = min(max(idx_fin_train + 1, int(0.80 * len(fechas_salida))), len(
48 fechas_salida) - 1)
49
50     fecha_corte_train = fechas_salida[idx_fin_train]
51     fecha_corte_test = fechas_salida[idx_fin_val]
52
53     df_train_raw = df_clean.filter(col("fecha_salida_dt") < lit(fecha_corte_train))
54     df_val_raw = df_clean.filter((col("fecha_salida_dt") >= lit(fecha_corte_train)) &
55 (col("fecha_salida_dt") < lit(fecha_corte_test)))
56     df_test_raw = df_clean.filter(col("fecha_salida_dt") >= lit(fecha_corte_test))

```

```

52     CLAVES_SERVICIO = [COL_ORIGEN, COL_DESTINO, "fecha_salida_dt"]
53     train_servicios = df_train_raw.select(*CLAVES_SERVICIO).dropDuplicates()
54     val_servicios = df_val_raw.select(*CLAVES_SERVICIO).dropDuplicates()
55     test_servicios = df_test_raw.select(*CLAVES_SERVICIO).dropDuplicates()
56
57     solapamiento_train_val = train_servicios.intersect(val_servicios).count()
58     solapamiento_train_test = train_servicios.intersect(test_servicios).count()
59     solapamiento_val_test = val_servicios.intersect(test_servicios).count()
60
61     def agregar_precios(df):
62         return (
63             df.groupBy(COL_ORIGEN, COL_DESTINO, "lead_time_dias", COL_TIPO_VEHICULO,
64             COL_CLASE, COL_TARIFA)
65             .agg(
66                 count("*").alias("n_observaciones_precio"),
67                 spark_round(avg(COL_PRECIO), 2).alias("precio_medio"),
68                 expr(f"percentile_approx({COL_PRECIO}, 0.5)").alias("precio_mediano"),
69                 spark_round(expr(f"stddev({COL_PRECIO})"), 2).alias("precio_std"),
70                 expr(f"max({COL_PRECIO})").alias("precio_maximo")
71             )
72         )
73
74     df_precios_train_pd = agregar_precios(df_train_raw).toPandas()
75     df_precios_val_pd = agregar_precios(df_val_raw).toPandas()
76     df_precios_test_pd = agregar_precios(df_test_raw).toPandas()
77
78     def crear_referencia_precio(df, origen="MADRID", destino="BARCELONA"):
79         ruta = df[(df[COL_ORIGEN] == origen) & (df[COL_DESTINO] == destino)].copy()
80         tarifa_maxima_historica = float(ruta["precio_maximo"].max())
81         ruta["precio_ponderado"] = ruta["precio_mediano"] * ruta["n_observaciones_precio"]
82
83         referencia = ruta.groupby("lead_time_dias")[["precio_ponderado", "n_observaciones_precio"]].sum()
84         referencia["precio_base"] = referencia["precio_ponderado"] / referencia["n_observaciones_precio"]
85         referencia = referencia.reset_index()[["lead_time_dias", "precio_base"]]
86
87         dias_completos = pd.DataFrame({"lead_time_dias": np.arange(0, MAX_DIAS + 1)})
88         referencia = dias_completos.merge(referencia, on="lead_time_dias", how="left")
89         precio_global = ruta["precio_ponderado"].sum() / ruta["n_observaciones_precio"].sum()
90         referencia["precio_base"] = referencia["precio_base"].interpolate().bfill().ffill().fillna(precio_global)
91         return referencia, tarifa_maxima_historica
92
93     referencia_precio_train, tarifa_max_train = crear_referencia_precio(df_precios_train_pd)
94     referencia_precio_val, tarifa_max_val = crear_referencia_precio(df_precios_val_pd)
95     referencia_precio_test, tarifa_max_test = crear_referencia_precio(df_precios_test_pd)
96
97     def crear_perfil_demanda(df_spark, origen="MADRID", destino="BARCELONA", demanda_media_objetivo=3.0):
98         ruta = df_spark.filter((col(COL_ORIGEN) == origen) & (col(COL_DESTINO) == destino))
99
100         actividad = ruta.groupBy("fecha_salida_dt", "lead_time_dias").agg(countDistinct(COL_FECHA_COMPRA).alias("n_snapshots"))

```

```
98     perfil = actividad.groupby("lead_time_dias").agg(avg("n_snapshots").alias("
actividad_media")).toPandas()
99     dias_completos = pd.DataFrame({"lead_time_dias": np.arange(1, MAX_DIAS + 1)})
100     perfil = dias_completos.merge(perfil, on="lead_time_dias", how="left")
101     perfil["actividad_media"] = perfil["actividad_media"].interpolate().bfill().ffill
().fillna(0.0)
102     perfil["urgencia"] = np.exp(-perfil["lead_time_dias"] / 15.0)
103     perfil["demanda_base"] = perfil["actividad_media"] * perfil["urgencia"]
104     media_demanda_base = perfil["demanda_base"].mean()
105     perfil["demanda_media"] = (perfil["demanda_base"] / media_demanda_base) *
demanda_media_objetivo
106     return perfil[["lead_time_dias", "demanda_media"]]
107
108     perfil_demanda_train = crear_perfil_demanda(df_train_raw)
109     perfil_demanda_val = crear_perfil_demanda(df_val_raw)
110     perfil_demanda_test = crear_perfil_demanda(df_test_raw)
111     ESCALA_PRECIO_TRAIN = float(tarifa_max_train)
112
```

A.3. Entorno de Simulación (Gymnasium) y Programación Dinámica

```

1      class RenfePricingEnv(gym.Env):
2          metadata = {"render_modes": []}
3
4          def __init__(self, referencia_precios, perfil_demanda, tarifa_maxima=300.0,
5              escala_precio=None, demanda_total_escenario=1.0, elasticidad=-0.8, capacidad=100,
6              max_dias=30, penalizacion_vacios=0.10, potencial_dp=None, coef_reward_shaping=0.0):
7              super().__init__()
8              self.referencia_precios = referencia_precios.copy()
9              self.perfil_demanda = perfil_demanda.copy()
10             self.tarifa_maxima = float(tarifa_maxima)
11             self.escala_precio = float(escala_precio if escala_precio is not None else
12                 tarifa_maxima)
13             self.demanda_total_escenario = float(demanda_total_escenario)
14             self.elasticidad = float(elasticidad)
15             self.capacidad = int(capacidad)
16             self.max_dias = int(max_dias)
17             self.penalizacion_vacios = float(penalizacion_vacios)
18             self.potencial_dp = potencial_dp
19             self.coef_reward_shaping = float(coef_reward_shaping)
20
21             self.multiplicadores = np.array([0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4],
22                 dtype=np.float32)
23             self.action_space = spaces.Discrete(len(self.multiplicadores))
24             self.observation_space = spaces.Box(low=np.zeros(3, dtype=np.float32), high=np.
25                 ones(3, dtype=np.float32), dtype=np.float32)
26             self.precio_por_dia = {int(dia): float(precio) for dia, precio in zip(self.
27                 referencia_precios["lead_time_dias"], self.referencia_precios["precio_base"])}
28             self.precio_fallback = float(self.referencia_precios["precio_base"].median())
29             self.demanda_por_dia = {int(dia): float(demanda) for dia, demanda in zip(self.
30                 perfil_demanda["lead_time_dias"], self.perfil_demanda["demanda_media"])}
31             self.factor_demanda = (self.demanda_total_escenario)
32             self.dias_restantes = self.max_dias
33             self.asientos_libres = self.capacidad
34
35             def precio_base(self, dias_restantes):
36                 return float(self.precio_por_dia.get(int(dias_restantes), self.precio_fallback))
37
38             def demanda_esperada(self, dias_restantes, action):
39                 precio_base = self.precio_base(dias_restantes)
40                 multiplicador = float(self.multiplicadores[action])
41                 precio_final = min(precio_base * multiplicador, self.tarifa_maxima)
42                 lambda_historica = self.demanda_por_dia.get(int(dias_restantes), 0.0)
43                 lambda_base = lambda_historica * self.factor_demanda
44                 ratio_precio = precio_final / max(precio_base, 1e-6)
45                 factor_precio = ratio_precio ** self.elasticidad
46                 lambda_demanda = lambda_base * factor_precio
47                 return max(float(lambda_demanda), 0.0)
48
49             def get_observacion(self):
50                 precio_base_actual = self.precio_base(self.dias_restantes)
51                 precio_base_norm = precio_base_actual / self.escala_precio
52                 precio_base_norm = float(np.clip(precio_base_norm, 0.0, 1.0))
53                 ocupacion = 1.0 - (self.asientos_libres / self.capacidad)
54                 return np.array([self.dias_restantes / float(self.max_dias), ocupacion,
55                     precio_base_norm], dtype=np.float32)

```

```

48
49     def reset(self, seed=None, options=None):
50         super().reset(seed=seed)
51         self.dias_restantes = self.max_dias
52         self.asientos_libres = self.capacidad
53         return (self.get_observacion(), {})
54
55     def valor_potencial(self, dias_restantes, asientos_libres):
56         if self.potencial_dp is None: return 0.0
57         dias = int(np.clip(dias_restantes, 0, self.max_dias))
58         asientos = int(np.clip(asientos_libres, 0, self.capacidad))
59         return float(self.potencial_dp[dias, asientos])
60
61     def step(self, action):
62         action = int(action)
63         potencial_antes = self.valor_potencial(self.dias_restantes, self.asientos_libres)
64         precio_base = self.precio_base(self.dias_restantes)
65         multiplicador = float(self.multiplicadores[action])
66         precio_final = min(precio_base * multiplicador, self.tarifa_maxima)
67         lambda_demanda = self.demanda_esperada(self.dias_restantes, action)
68         demanda_real = int(self.np_random.poisson(lambda_demanda))
69         entradas_vendidas = min(demanda_real, self.asientos_libres)
70
71         self.asientos_libres -= entradas_vendidas
72         ingreso_paso = float(entradas_vendidas * precio_final)
73         self.dias_restantes -= 1
74
75         terminated = bool(self.dias_restantes <= 0 or self.asientos_libres <= 0)
76         truncated = False
77         penalizacion = 0.0
78         if terminated and self.asientos_libres > 0:
79             penalizacion = float(self.asientos_libres * self.precio_fallback * self.
penalizacion_vacios)
80
81         reward_euros = ingreso_paso - penalizacion
82         potencial_despues = self.valor_potencial(self.dias_restantes, self.
asientos_libres)
83         reward_shaping_euros = self.coef_reward_shaping * (potencial_despues -
potencial_antes)
84         reward_total_euros = reward_euros + reward_shaping_euros
85         reward = reward_total_euros / self.escala_precio
86
87         info = {
88             "precio_base": precio_base, "precio_final": precio_final, "multiplicador":
multiplicador,
89             "demanda_esperada": lambda_demanda, "demanda_real": demanda_real, "
entradas_vendidas": entradas_vendidas,
90             "ingreso_paso": ingreso_paso, "penalizacion_final": penalizacion, "
reward_euros": reward_euros,
91             "potencial_antes": potencial_antes, "potencial_despues": potencial_despues, "
reward_shaping_euros": reward_shaping_euros,
92             "reward_total_euros": reward_total_euros, "asientos_libres": self.
asientos_libres,
93         }
94         return (self.get_observacion(), float(reward), terminated, truncated, info)
95
96     def calcular_optimo_dp(entorno):
97         max_dias = entorno.max_dias
98         capacidad = entorno.capacidad

```

```
99     acciones = entorno.multiplicadores
100     V = np.zeros((max_dias + 1, capacidad + 1), dtype=np.float64)
101     politica = np.zeros((max_dias + 1, capacidad + 1), dtype=np.int32)
102     for c in range(capacidad + 1):
103         V[0, c] = -c * entorno.precio_fallback * entorno.penalizacion_vacios
104
105     for d in range(1, max_dias + 1):
106         for c in range(capacidad + 1):
107             if c == 0:
108                 V[d, c] = 0.0
109                 politica[d, c] = 0
110                 continue
111             valores = np.zeros(len(acciones))
112             for a in range(len(acciones)):
113                 precio_base = entorno.precio_base(d)
114                 multiplicador = float(acciones[a])
115                 precio = min(precio_base * multiplicador, entorno.tarifa_maxima)
116                 lam = entorno.demanda_esperada(d, a)
117                 valor = 0.0
118                 if lam > 0:
119                     dem_array = np.arange(c)
120                     probs = poisson.pmf(dem_array, lam)
121                     valores_futuros = V[d - 1, c - dem_array]
122                     valor += np.sum(probs * (dem_array * precio + valores_futuros))
123                     prob_agotar = poisson.sf(c - 1, lam)
124                     valor += prob_agotar * (c * precio + V[d - 1, 0])
125                 else:
126                     valor += V[d - 1, c]
127                 valores[a] = valor
128             politica[d, c] = int(np.argmax(valores))
129             V[d, c] = float(np.max(valores))
130         return V, politica
131
```

A.4. Baseline de Machine Learning (Random Forest)

```

1      def crear_dataset_rf_real(df_spark, n_muestras=50000, seed=SEED):
2          rng = np.random.default_rng(seed)
3          filas = []
4          rutas = df_spark.select(COL_ORIGEN, COL_DESTINO, "lead_time_dias",
COL_TIPO_VEHICULO, COL_CLASE, COL_TARIFA, COL_PRECIO).distinct().toPandas()
5
6          for _ in range(n_muestras):
7              fila = rutas.sample(1).iloc[0]
8              dias = int(fila["lead_time_dias"])
9              origen = fila[COL_ORIGEN]
10             destino = fila[COL_DESTINO]
11             precio_base = float(fila[COL_PRECIO])
12             asientos = int(rng.integers(20, CAPACIDAD + 1))
13             multiplicador = float(rng.choice(np.array([0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2,
1.3, 1.4])))
14             precio_final = min(precio_base * multiplicador, 300.0)
15             demanda_latente = max(1.0, (asientos * 0.55) * (1.0 / max(dias, 1)) *
multiplicador)
16             demanda_sintetica = int(rng.poisson(demanda_latente))
17             ventas_sinteticas = min(demanda_sintetica, asientos)
18             filas.append({
19                 "dias_restantes": dias, "asientos_libres": asientos, "multiplicador":
multiplicador,
20                 "precio_base": precio_base, "precio_final": precio_final, "ventas_sinteticas
": ventas_sinteticas,
21                 "origen": origen, "destino": destino,
22             })
23             return pd.DataFrame(filas)
24
25             df_rf_real = crear_dataset_rf_real(df_train_raw, n_muestras=50_000, seed=SEED)
26             COLUMNAS_RF = ["dias_restantes", "asientos_libres", "multiplicador", "precio_base
"]
27             X_rf = df_rf_real[COLUMNAS_RF]
28             y_rf = df_rf_real["ventas_sinteticas"]
29
30             modelo_rf = RandomForestRegressor(n_estimators=400, min_samples_leaf=4,
random_state=SEED, n_jobs=-1)
31             modelo_rf.fit(X_rf, y_rf)
32
33             CACHE_ACCIONES_RF = {}
34
35             def elegir_accion_rf(estado, entorno):
36                 estado_array = np.asarray(estado, dtype=np.float32)
37                 dias = int(np.round(estado_array[0] * entorno.max_dias))
38                 ocupacion = float(estado_array[1])
39                 asientos = int(np.round((1.0 - ocupacion) * entorno.capacidad))
40
41                 if dias <= 0 or asientos <= 0:
42                     return 4
43
44                 precio_base = entorno.precio_base(dias)
45                 clave = (dias, asientos, round(precio_base, 2))
46                 if clave in CACHE_ACCIONES_RF:
47                     return CACHE_ACCIONES_RF[clave]
48
49                 multiplicadores = entorno.multiplicadores
50                 candidatos = pd.DataFrame({

```

```
51         "dias_restantes": np.full(len(multiplicadores), dias),
52         "asientos_libres": np.full(len(multiplicadores), asientos),
53         "multiplicador": multiplicadores,
54         "precio_base": np.full(len(multiplicadores), precio_base)
55     })
56
57     demanda_predicha = modelo_rf.predict(candidatos)
58     ventas_predichas = np.minimum(np.maximum(demanda_predicha, 0), asientos)
59     precios_reales = np.minimum(precio_base * multiplicadores, entorno.tarifa_maxima)
60     ingresos_esperados = ventas_predichas * precios_reales
61     mejor_accion = int(np.argmax(ingresos_esperados))
62     CACHE_ACCIONES_RF[clave] = mejor_accion
63     return mejor_accion
64
```

A.5. Entrenamiento del Agente PPO y Callbacks

```

1      from stable_baselines3.common.callbacks import BaseCallback, CallbackList
2      import torch as th
3
4      SEMILLAS_PPO = [42, 43, 44, 45, 46]
5      TIMESTEPS_PPO = 200000
6
7      CONFIGURACIONES_PPO = [
8          {"nombre": "gamma_099_exploracion_moderada", "learning_rate": 3e-4, "ent_coef":
0.03, "n_steps": 1024, "batch_size": 256, "gae_lambda": 0.95, "gamma": 0.99, "
coef_reward_shaping": 0.02},
9          {"nombre": "gamma_099_exploracion_alta", "learning_rate": 1e-4, "ent_coef": 0.05,
"n_steps": 1024, "batch_size": 256, "gae_lambda": 0.95, "gamma": 0.99, "
coef_reward_shaping": 0.02},
10         {"nombre": "gamma_100_shaping_estable", "learning_rate": 1e-4, "ent_coef": 0.02,
"n_steps": 2048, "batch_size": 256, "gae_lambda": 0.95, "gamma": 1.0, "
coef_reward_shaping": 0.02},
11     ]
12
13     class EntropiaPPOCallback(BaseCallback):
14     def __init__(self, estados_diagnostico, frecuencia=10_000, verbose=0):
15     super().__init__(verbose)
16     self.estados_diagnostico = estados_diagnostico
17     self.frecuencia = frecuencia
18     self.historial = []
19
20     def _on_step(self):
21     if self.num_timesteps % self.frecuencia == 0:
22     obs = th.as_tensor(self.estados_diagnostico, dtype=th.float32, device=self.model.
device)
23     with th.no_grad():
24     distribucion = self.model.policy.get_distribution(obs)
25     entropia = float(distribucion.entropy().mean().cpu().item())
26     self.historial.append({"timesteps": int(self.num_timesteps), "entropia_media":
entropia})
27     return True
28
29     def entrenar_ppo(semilla, configuracion):
30     set_random_seed(semilla)
31     nombre = configuracion["nombre"]
32     env_train = crear_entorno_train(configuracion["coef_reward_shaping"])
33     env_val = crear_entorno_validacion()
34
35     try:
36     callback_eval = EvalCallback(env_val, best_model_save_path=str(carpeta_modelo),
log_path=str(carpeta_logs), eval_freq=10_000, n_eval_episodes=50, deterministic=True,
render=False)
37     callback_entropia = EntropiaPPOCallback(ESTADOS_DIAGNOSTICO, frecuencia=10_000)
38     callback = CallbackList([callback_eval, callback_entropia])
39
40     modelo = PPO("MlpPolicy", env_train, learning_rate=configuracion["learning_rate
"], n_steps=configuracion["n_steps"], batch_size=configuracion["batch_size"], gamma=
configuracion["gamma"], gae_lambda=configuracion["gae_lambda"], policy_kwargs={"
net_arch": {"pi": [128, 128], "vf": [128, 128]}}, seed=semilla, ent_coef=
configuracion["ent_coef"], verbose=0, tensorboard_log=str(RUTA_LOGS))
41     modelo.learn(total_timesteps=TIMESTEPS_PPO, callback=callback, tb_log_name=f"
PPO_revisado_{nombre}_seed_{semilla}")
42

```

```
43     evaluaciones = np.load(ruta_evaluaciones)
44     mejores_rewards = evaluaciones["results"].mean(axis=1)
45     mejor_reward_val = float(mejores_rewards.max())
46     mejor_modelo = PPO.load(str(ruta_mejor))
47
48     return mejor_modelo, mejor_reward_val, callback_entropia.historial
49     finally:
50         env_train.close()
51         env_val.close()
52
```

A.6. Validación de Negocio (Monte Carlo) y Generación de Gráficos

```

1  ESCENARIOS_DEMANDA = {"Baja": 0.80, "Media": 1.00, "Alta": 1.20}
2  ESCENARIOS_ELASTICIDAD = {"Menos sensible": -0.4, "Nominal": -1.2, "Más sensible
   ": -1.8}
3  N_EVALUACIONES = 1_000
4
5  def accion_heuristica(estado, entorno):
6  dias = int(np.clip(np.round(estado[0] * entorno.max_dias), 0, entorno.max_dias))
7  ocupacion = float(np.clip(estado[1], 0.0, 1.0))
8  if dias <= 10 and ocupacion >= 0.80: return 8
9  if dias >= 20 and ocupacion <= 0.40: return 2
10 return 4
11
12 def ejecutar_episodio(estrategia, entorno, semilla, modelo_ppo=None, politica_dp=
   None):
13 estado, _ = entorno.reset(seed=semilla)
14 terminado = False
15 ingreso_total = 0.0
16 penalizacion_total = 0.0
17 reward_total = 0.0
18 acciones = []
19
20 while not terminado:
21 dias = int(np.clip(np.round(estado[0] * entorno.max_dias), 0, entorno.max_dias))
22 asientos = int(np.clip(np.round((1.0 - float(estado[1])) * entorno.capacidad), 0,
   entorno.capacidad))
23
24 if estrategia == "Política base (1.0x)": accion = 4
25 elif estrategia == "Heurística": accion = accion_heuristica(estado, entorno)
26 elif estrategia == "RF sintético miope": accion = elegir_accion_rf(estado,
   entorno)
27 elif estrategia == "DP oracle / techo teórico": accion = int(politica_dp[dias,
   asientos])
28 elif estrategia == "PP0":
29 accion_predicha, _ = modelo_ppo.predict(estado, deterministic=True)
30 accion = int(np.asarray(accion_predicha).item())
31
32 acciones.append(accion)
33 estado, reward, terminado, truncado, info = entorno.step(accion)
34 ingreso_total += float(info["ingreso_paso"])
35 penalizacion_total += float(info["penalizacion_final"])
36 reward_total += float(reward)
37
38 objetivo_total = ingreso_total - penalizacion_total
39 ocupacion_final_pct = 100.0 * (entorno.capacidad - entorno.asientos_libres) /
   entorno.capacidad
40 multiplicador_medio = float(np.mean([entorno.multiplicadores[a] for a in acciones
   ])) if acciones else 1.0
41
42 return {"ingreso_total": ingreso_total, "penalizacion_total": penalizacion_total,
   "objetivo_total": objetivo_total, "reward_total": reward_total, "ocupacion_final_pct
   ": ocupacion_final_pct, "multiplicador_medio": multiplicador_medio}
43
44 # ... (Bucle iterativo de recoleccion y simulacion Monte Carlo para 1.000
   episodios) ...
45

```

```
46     # Graficado con Seaborn y Matplotlib de los resultados
47     import seaborn as sns
48     import matplotlib.pyplot as plt
49
50     plt.figure(figsize=(12, 6))
51     sns.boxplot(data=df_resultados, x="Estrategia", y="objetivo_total")
52     plt.xticks(rotation=20)
53     plt.title("Distribución del objetivo total por estrategia")
54     plt.show()
55
56     if 'politica_dp_ruta' in locals():
57         plt.figure(figsize=(12, 7))
58         plt.imshow(politica_dp_ruta[1:, 1:][:, :-1, :], aspect="auto")
59         plt.xlabel("Asientos libres")
60         plt.ylabel("Días restantes")
61         plt.title(f"Mapa de política DP (Ejemplo: {nombre_ruta})")
62         plt.colorbar(label="Índice de acción")
63         plt.show()
64
```

A.7. Análisis de la Política, Heatmap PPO y Exportación MLOps

```

1     politica_ppo = np.zeros((MAX_DIAS + 1, CAPACIDAD + 1), dtype=int)
2     estados_batch = []
3     coordenadas = []
4
5     for dias in range(1, MAX_DIAS + 1):
6         precio_base = env_dp.precio_base(dias)
7         precio_norm = float(np.clip(precio_base / ESCALA_PRECIO_TRAIN, 0.0, 1.0))
8         for asientos_libres in range(1, CAPACIDAD + 1):
9             ocupacion = 1.0 - (asientos_libres / CAPACIDAD)
10            estados_batch.append([dias / MAX_DIAS, ocupacion, precio_norm])
11            coordenadas.append((dias, asientos_libres))
12
13            estados_array = np.array(estados_batch, dtype=np.float32)
14            acciones_batch, _ = modelo_ppo_best.predict(estados_array, deterministic=True)
15
16            for (dias, asientos_libres), accion in zip(coordenadas, acciones_batch):
17                politica_ppo[dias, asientos_libres] = int(accion)
18
19            # Visualizacion del Heatmap de la politica PPO
20            plt.figure(figsize=(12, 7))
21            plt.imshow(politica_util[:, :-1, :], aspect="auto")
22            plt.xlabel("Asientos libres")
23            plt.ylabel("Días restantes")
24            plt.title("Mapa de política PPO seleccionada")
25            plt.colorbar(label="Índice de acción")
26            plt.show()
27
28            # Bloque de Exportación
29            cerebro_multiruta = {}
30
31            for origen, destino in rutas_disponibles:
32                referencia_ruta, tarifa_max_ruta = crear_referencia_precio(df_completo_desarrollo
, origen=origen, destino=destino)
33                precios_base_diarios = {str(int(fila["lead_time_dias"])): float(fila["precio_base
"]) for _, fila in referencia_ruta.iterrows()}
34
35                estados_export = []
36                coords_export = []
37                for dias in range(0, MAX_DIAS + 1):
38                    precio_base_dia = precios_base_diarios.get(str(dias), float(
referencia_precio_train[referencia_precio_train["lead_time_dias"] == dias]["
precio_base"].iloc[0]))
39                    precio_norm = float(np.clip(precio_base_dia / ESCALA_PRECIO_TRAIN, 0.0, 1.0))
40                    for asientos_libres in range(0, CAPACIDAD + 1):
41                        ocupacion = 1.0 - (asientos_libres / CAPACIDAD)
42                        estados_export.append([dias / MAX_DIAS, ocupacion, precio_norm])
43                        coords_export.append(f"{dias}_{asientos_libres}")
44
45                    estados_array_exp = np.array(estados_export, dtype=np.float32)
46                    acciones_exp, _ = modelo_ppo_best.predict(estados_array_exp, deterministic=True)
47                    matriz_ruta = {coord: int(acc) for coord, acc in zip(coords_export, acciones_exp)
}
48
49                    clave_ruta = f"{origen}_{destino}".upper().replace(" ", "_")
50
51                    cerebro_multiruta[clave_ruta] = {
52                        "politica": matriz_ruta,
53                        "precios_base": precios_base_diarios,

```

```
53         "tarifa_maxima": float(tarifa_max_ruta),
54         "escala_precio": float(ESCALA_PRECIO_TRAIN),
55     }
56
57     with open(ruta_export / "cerebro_ia.json", "w", encoding="utf-8") as archivo:
58         json.dump(cerebro_multiruta, archivo, ensure_ascii=False, indent=2)
59
```

A.8. Arquitectura Guardrails y Alertas en Vanilla JS

```
1      // Intercepcion y evaluacion determinista de la politica DRL
2      const pCrudo = pBase * mult;
3      const veto = pCrudo > max;
4      const pFinal = veto ? max : pCrudo; // Truncamiento corporativo de seguridad
5
6      // Manipulacion del DOM para alertas visuales al analista
7      if (veto) {
8          D.kpiVarTexto.innerHTML = '<span class="text-rose-600 font-bold">LIMITE
APLICADO</span>';
9          D.bannerGob.classList.remove("hidden");
10     } else {
11         D.bannerGob.classList.add("hidden");
12     }
13
```